

Healthcare Insurance Fraud Detection and Risk Analytics

Project Overview

Healthcare fraud is one of the most significant challenges faced by insurance providers and healthcare organizations, resulting in billions of dollars in financial losses every year. Fraudulent claims increase operational costs, reduce system efficiency, and place additional financial burdens on both insurers and patients.

This project aims to analyze healthcare insurance claims data to uncover fraud patterns, identify high-risk providers and claim behaviors, and develop a machine learning model capable of predicting potentially fraudulent claims. The project combines exploratory data analysis, statistical investigation, feature engineering, and predictive modeling to provide actionable insights for fraud prevention and risk management.

By leveraging data-driven techniques, this project seeks to improve fraud detection capabilities, reduce financial losses, and support more effective allocation of investigative resources.

Project Objectives

Business Objectives

Identify key drivers contributing to healthcare claim fraud.

Quantify the financial impact of fraudulent claims.

Discover high-risk providers, specialties, and claim characteristics.

Understand behavioral patterns associated with fraudulent activity.

Support fraud investigation teams through data-driven insights.

Improve operational efficiency by prioritizing high-risk claims.

Analytical Objective

Perform comprehensive exploratory data analysis (EDA).

Investigate relationships between patient, provider, financial, and claim-related variables.

Conduct statistical analyses to validate fraud-related patterns.

Engineer meaningful features that improve fraud detection performance.

Build and evaluate machine learning models for fraud prediction.

Interpret model decisions using explainable AI techniques.

Machine Learning Objectives

Predict whether a healthcare claim is fraudulent or legitimate.

Address class imbalance in fraud detection.

Compare multiple classification algorithms.

Identify the most influential factors contributing to fraud.

Generate fraud risk scores for future claims.

Dataset Description

The dataset contains 18,764 healthcare insurance claims designed to simulate realistic healthcare billing and fraud scenarios.

Target Variable

Is_Fraud

0 = Legitimate Claim 1 = Fraudulent Claim

Dataset Features

Provider Information

Provider_ID

Provider_Specialty

Number_of_Claims_Per_Provider_Monthly

Patient Information

Patient_Age

Patient_Gender

Patient_State

Chronic_Condition_Flag

Prior_Visits_12m

Financial Information

Claim_Amount

Approved_Amount

Insurance_Type

Claim_Status

Medical Information

Diagnosis_Code

Procedure_Code

Length_of_Stay

Visit_Type

Temporal Information

Claim_Submission_Date

Days_Between_Service_and_Claim

These features provide valuable insights into provider behavior, patient characteristics, claim patterns, and financial risk factors associated with healthcare fraud.

```
In [5]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats
from scipy.stats import chi2_contingency
from scipy.stats import mannwhitneyu
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    roc_auc_score,
    confusion_matrix,
    classification_report
)
from imblearn.over_sampling import SMOTE
from xgboost import XGBClassifier
import shap
import warnings
warnings.filterwarnings('ignore')
```

```
In [6]: df = pd.read_csv(r"C:\Users\papus\OneDrive\Documents\HealthCare Fraud Detection\Health_Care_Fraud_Detection.csv")
```

```
In [7]: df.head()
```

Out[7]:

	Provider_ID	Claim_ID	Patient_Age	Patient_Gender	Diagnosis_Code	Procedure_Code	Claim_Amount	Approved_Amount	Insurance_Type	Claim_Subn
0	P0052	C0000000	37	Male	I25.10	36415	443.51	393.16	Medicaid	
1	P0121	C0000001	21	Female	E11.9	99213	467.50	461.33	Self-Pay	
2	P0140	C0000002	78	Female	J06.9	93000	591.69	530.06	Medicaid	
3	P0202	C0000003	65	Male	I10	93000	235.15	189.11	Private	
4	P0135	C0000004	36	Male	M54.5	85025	487.96	369.91	Private	

In [9]: df.tail()

Out[9]:

	Provider_ID	Claim_ID	Patient_Age	Patient_Gender	Diagnosis_Code	Procedure_Code	Claim_Amount	Approved_Amount	Insurance_Type	Claim_
18759	P5060	C0018759	17	Male	J06.9	87086	611.59	473.77	Medicare	
18760	P5765	C0018760	53	Female	F41.9	93000	694.95	577.76	Medicare	
18761	P4612	C0018761	44	Male	E11.9	97110	1059.07	893.63	Medicare	
18762	P7553	C0018762	38	Female	E78.5	71046	860.95	604.64	Private	
18763	P8378	C0018763	46	Male	M54.5	97110	1017.03	1022.37	Medicare	

In [10]: df.describe()

Out[10]:

	Patient_Age	Procedure_Code	Claim_Amount	Approved_Amount	Days_Between_Service_and_Claim	Number_of_Claims_Per_Provider_Monthly	
count	18764.000000	18764.000000	18764.000000	18764.000000	18764.000000	18764.000000	1876
mean	49.495843	86924.101418	633.281776	530.626986	14.209977	68.442123	
std	17.915047	15014.580425	1910.742702	1583.905123	8.388061	14.788624	
min	1.000000	36415.000000	60.000000	50.000000	0.000000	42.000000	
25%	37.000000	80053.000000	301.770000	252.777500	7.000000	59.000000	
50%	50.000000	93000.000000	508.490000	424.670000	14.000000	66.000000	
75%	62.000000	99213.000000	788.932500	661.757500	21.000000	75.000000	
max	95.000000	99214.000000	95251.000000	73867.640000	29.000000	144.000000	

In [11]:

df.info()

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 18764 entries, 0 to 18763
```

```
Data columns (total 20 columns):
```

#	Column	Non-Null Count	Dtype
0	Provider_ID	18764 non-null	object
1	Claim_ID	18764 non-null	object
2	Patient_Age	18764 non-null	int64
3	Patient_Gender	18764 non-null	object
4	Diagnosis_Code	18764 non-null	object
5	Procedure_Code	18764 non-null	int64
6	Claim_Amount	18764 non-null	float64
7	Approved_Amount	18764 non-null	float64
8	Insurance_Type	18164 non-null	object
9	Claim_Submission_Date	18764 non-null	object
10	Days_Between_Service_and_Claim	18764 non-null	int64
11	Number_of_Claims_Per_Provider_Monthly	18764 non-null	int64
12	Provider_Specialty	18101 non-null	object
13	Patient_State	18764 non-null	object
14	Claim_Status	18764 non-null	object
15	Is_Fraud	18764 non-null	int64
16	Length_of_Stay	18764 non-null	int64
17	Visit_Type	18764 non-null	object
18	Chronic_Condition_Flag	18764 non-null	int64
19	Prior_Visits_12m	17936 non-null	float64

```
dtypes: float64(3), int64(7), object(10)
```

```
memory usage: 2.9+ MB
```

```
In [13]: df['Claim_Submission_Date'] = pd.to_datetime(  
         df['Claim_Submission_Date'],  
         format='%d-%m-%Y'  
         )
```

```
In [14]: cat_cols = [  
         'Provider_ID',  
         'Patient_Gender',  
         'Diagnosis_Code',  
         'Insurance_Type',  
         'Provider_Specialty',  
         'Patient_State',  
         'Claim_Status',  
         'Visit_Type'  
         ]
```

```
for col in cat_cols:
    df[col] = df[col].astype('category')
```

```
In [16]: df.isnull().sum()
```

```
Out[16]: Provider_ID          0
        Claim_ID            0
        Patient_Age         0
        Patient_Gender       0
        Diagnosis_Code       0
        Procedure_Code       0
        Claim_Amount         0
        Approved_Amount      0
        Insurance_Type       600
        Claim_Submission_Date 0
        Days_Between_Service_and_Claim 0
        Number_of_Claims_Per_Provider_Monthly 0
        Provider_Specialty    663
        Patient_State         0
        Claim_Status          0
        Is_Fraud              0
        Length_of_Stay        0
        Visit_Type            0
        Chronic_Condition_Flag 0
        Prior_Visits_12m      828
        dtype: int64
```

```
In [19]: df[['Insurance_Type', 'Prior_Visits_12m']].isnull().sum()
```

```
Out[19]: Insurance_Type      600
        Prior_Visits_12m     828
        dtype: int64
```

```
In [20]: (df[['Insurance_Type', 'Prior_Visits_12m']].isnull().sum()/len(df))*100
```

```
Out[20]: Insurance_Type      3.197612
        Prior_Visits_12m     4.412705
        dtype: float64
```

```
In [21]: df['Insurance_Type'].value_counts(dropna=False)
```



```
Out[21]: Insurance_Type
Medicare    4815
Private     4475
Self-Pay    4464
Medicaid   4410
NaN          600
Name: count, dtype: int64
```

```
In [22]: df['Insurance_Type'].fillna(
        df['Insurance_Type'].mode()[0],
        inplace=True
    )
```

```
In [23]: df[['Insurance_Type']].isnull().sum()
```

```
Out[23]: Insurance_Type    0
dtype: int64
```

```
In [24]: df['Prior_Visits_12m'].describe()
```

```
Out[24]: count    17936.000000
mean           3.031477
std            1.691256
min            0.000000
25%            2.000000
50%            3.000000
75%            4.000000
max           12.000000
Name: Prior_Visits_12m, dtype: float64
```

```
In [26]: df['Prior_Visits_12m'].median()
```

```
Out[26]: np.float64(3.0)
```

```
In [27]: df['Prior_Visits_12m'].mean()
```

```
Out[27]: np.float64(3.031477475468332)
```

```
In [29]: df['Prior_Visits_12m'] = df['Prior_Visits_12m'].fillna(
        df['Prior_Visits_12m'].median()
    )
```

```
In [30]: df[['Prior_Visits_12m']].isnull().sum()
```

```
Out[30]: Prior_Visits_12m    0  
dtype: int64
```

```
In [31]: df.duplicated().sum()
```

```
Out[31]: np.int64(0)
```

Target Variable Analysis

```
In [32]: df['Is_Fraud']
```

```
Out[32]: 0      0  
         1      0  
         2      0  
         3      0  
         4      0  
         ..  
18759    0  
18760    0  
18761    0  
18762    1  
18763    0  
Name: Is_Fraud, Length: 18764, dtype: int64
```

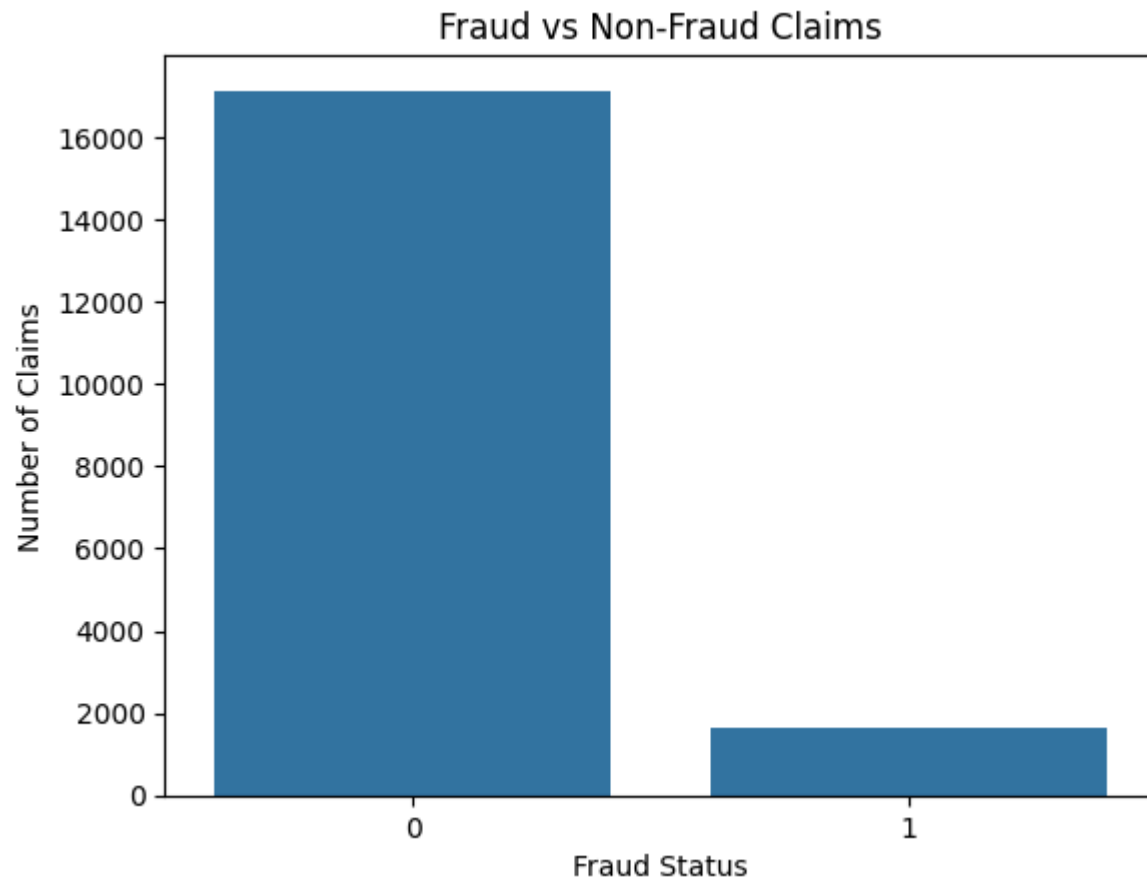
```
In [36]: # Count  
df['Is_Fraud'].value_counts()
```

```
Out[36]: Is_Fraud  
0      17139  
1       1625  
Name: count, dtype: int64
```

```
In [37]: # Percentage  
df['Is_Fraud'].value_counts(normalize=True)*100
```

```
Out[37]: Is_Fraud  
0      91.3398  
1       8.6602  
Name: proportion, dtype: float64
```

```
In [38]: sns.countplot(x='Is_Fraud', data=df)
plt.title('Fraud vs Non-Fraud Claims')
plt.xlabel('Fraud Status')
plt.ylabel('Number of Claims')
plt.show()
```



Healthcare fraud represents a relatively small proportion of total claims but can account for a disproportionately large share of financial losses. This imbalance mirrors real-world insurance environments where fraudulent activities are hidden within large volumes of legitimate transactions.

```
In [39]: num_cols = [
    'Patient_Age',
    'Claim_Amount',
    'Approved_Amount',
    'Prior_Visits_12m',
    'Length_of_Stay',
```

```

'Days_Between_Service_and_Claim',
'Number_of_Claims_Per_Provider_Monthly'
]

df[num_cols].describe().T

```

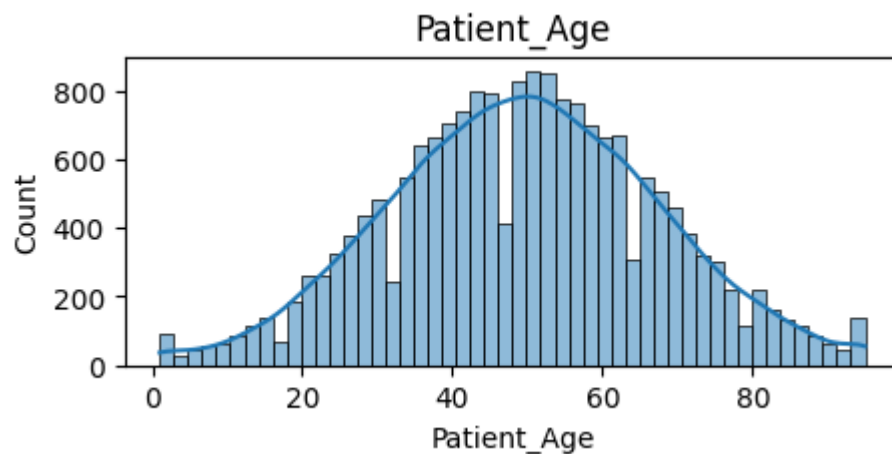
Out[39]:

	count	mean	std	min	25%	50%	75%	max
Patient_Age	18764.0	49.495843	17.915047	1.0	37.0000	50.00	62.0000	95.00
Claim_Amount	18764.0	633.281776	1910.742702	60.0	301.7700	508.49	788.9325	95251.00
Approved_Amount	18764.0	530.626986	1583.905123	50.0	252.7775	424.67	661.7575	73867.64
Prior_Visits_12m	18764.0	3.030088	1.653530	0.0	2.0000	3.00	4.0000	12.00
Length_of_Stay	18764.0	2.199371	1.704916	0.0	1.0000	2.00	3.0000	5.00
Days_Between_Service_and_Claim	18764.0	14.209977	8.388061	0.0	7.0000	14.00	21.0000	29.00
Number_of_Claims_Per_Provider_Monthly	18764.0	68.442123	14.788624	42.0	59.0000	66.00	75.0000	144.00

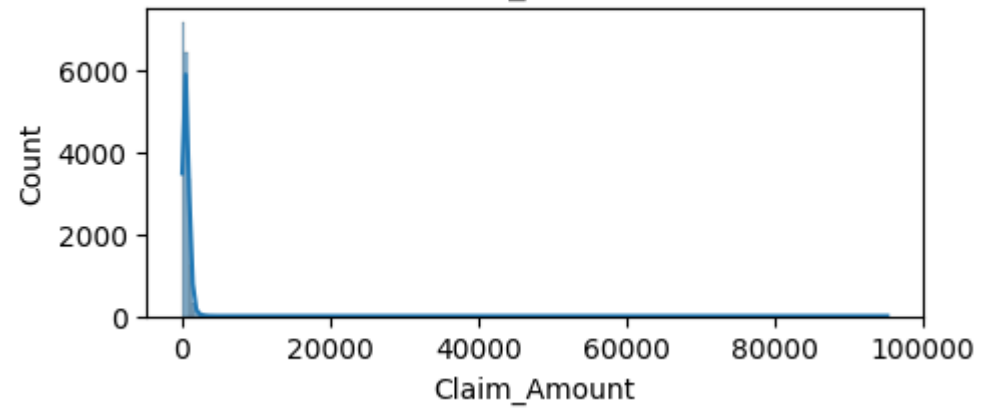
```

In [41]: for col in num_cols:
plt.figure(figsize=(5,2))
sns.histplot(df[col], kde=True)
plt.title(col)
plt.show()

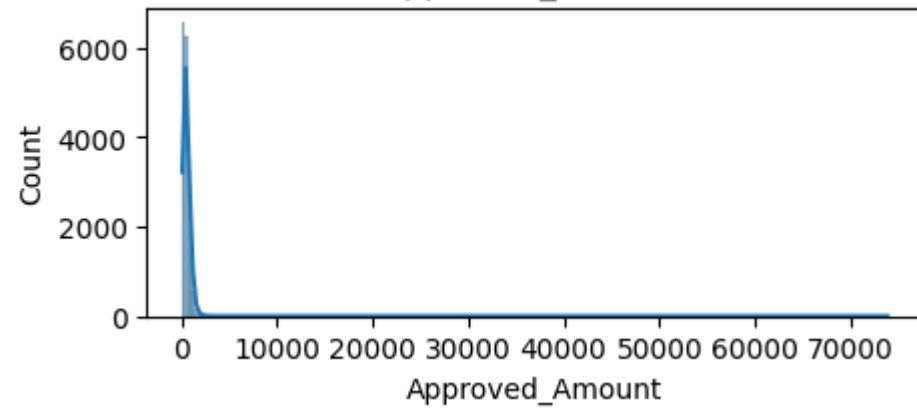
```



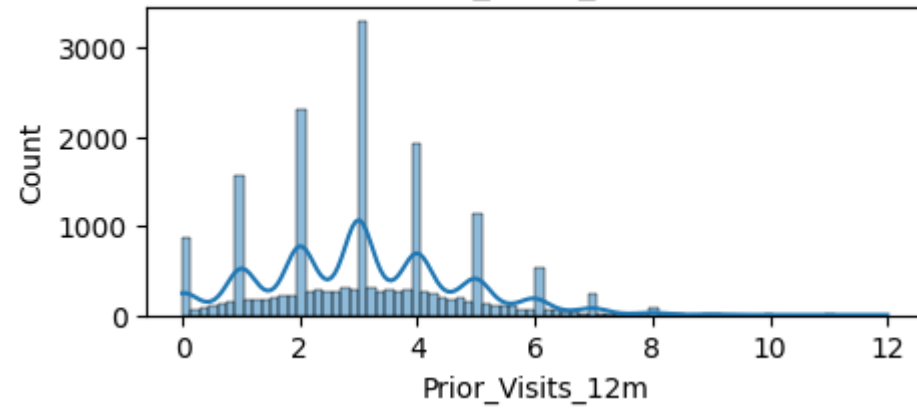
Claim_Amount

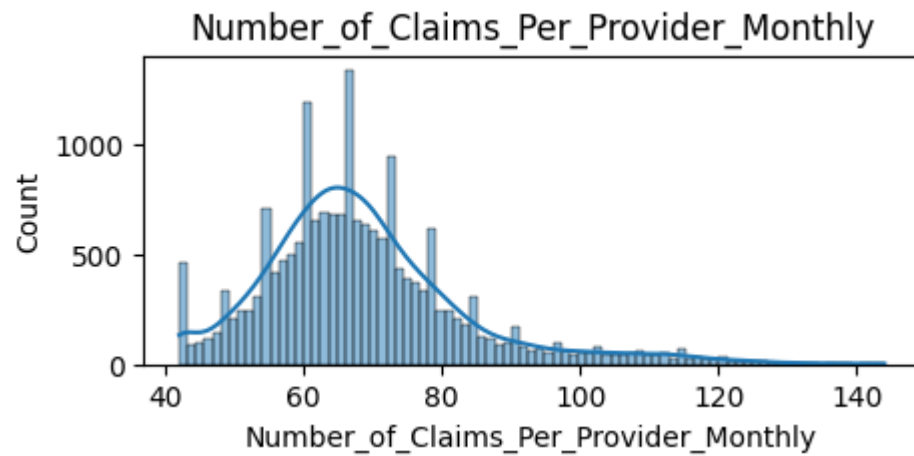
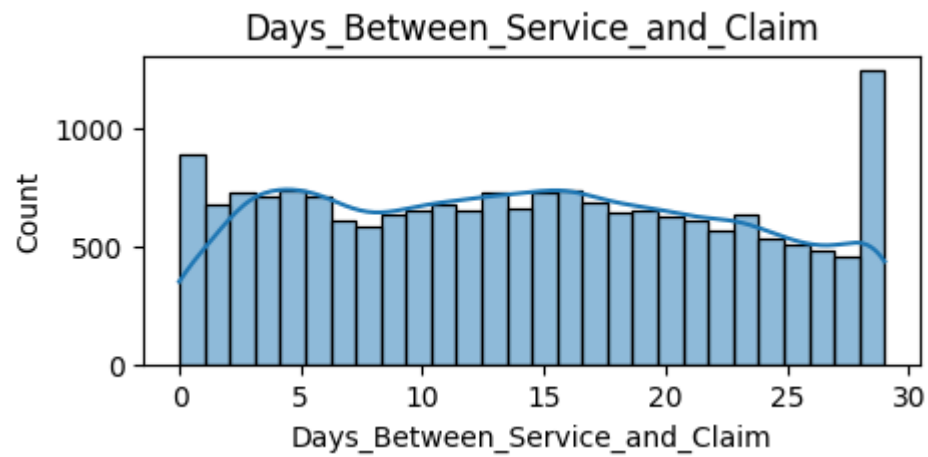
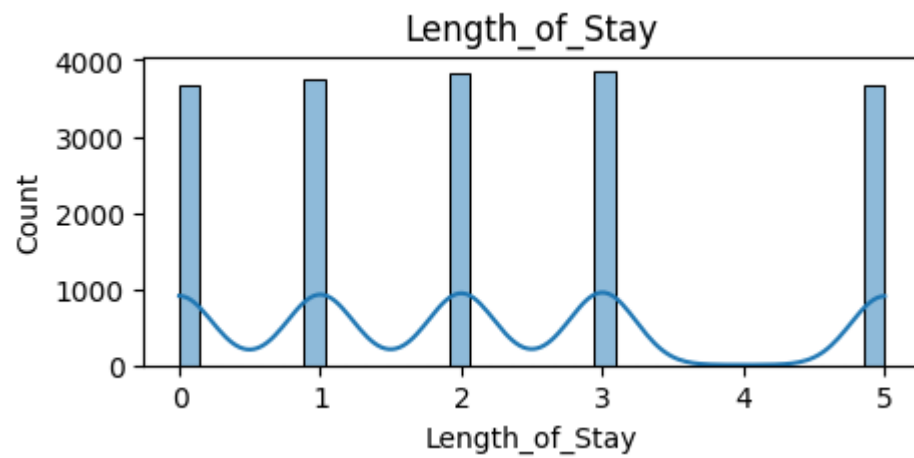


Approved_Amount



Prior_Visits_12m





```
In [42]: df['Insurance_Type'].value_counts()
```

```
Out[42]: Insurance_Type
Medicare      5415
Private       4475
Self-Pay      4464
Medicaid     4410
Name: count, dtype: int64
```

```
In [43]: df['Provider_Specialty'].value_counts()
```

```
Out[43]: Provider_Specialty
Internal Medicine    4053
General Practice     3459
Orthopedics          2909
Neurology            2851
Cardiology           2519
Pulmonology          2310
Name: count, dtype: int64
```

```
In [44]: df['Patient_Gender'].value_counts()
```

```
Out[44]: Patient_Gender
Male      9930
Female    8834
Name: count, dtype: int64
```

```
In [45]: df['Visit_Type'].value_counts()
```

```
Out[45]: Visit_Type
Outpatient    6444
Emergency     6235
Inpatient     6085
Name: count, dtype: int64
```

```
In [46]: df['Claim_Status'].value_counts()
```

```
Out[46]: Claim_Status
Approved    12705
Rejected     3691
Pending     2368
Name: count, dtype: int64
```

```
In [47]: df['Chronic_Condition_Flag'].value_counts()
```

```
Out[47]: Chronic_Condition_Flag
0      13323
1       5441
Name: count, dtype: int64
```

```
In [48]: df['Insurance_Type'].value_counts(normalize=True)*100
```

```
Out[48]: Insurance_Type
Medicare      28.858452
Private       23.848860
Self-Pay      23.790237
Medicaid     23.502452
Name: proportion, dtype: float64
```

```
In [49]: num_cols = [
    'Patient_Age',
    'Claim_Amount',
    'Approved_Amount',
    'Prior_Visits_12m',
    'Length_of_Stay',
    'Days_Between_Service_and_Claim',
    'Number_of_Claims_Per_Provider_Monthly'
]
```

```
In [50]: for col in num_cols:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1

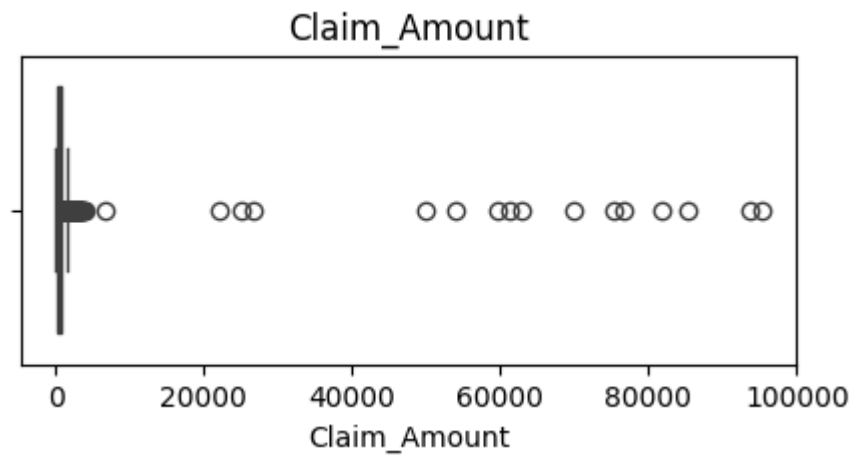
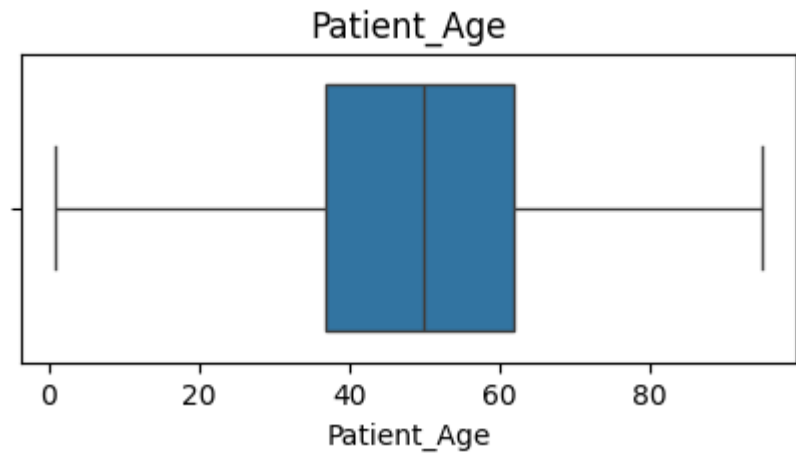
    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR

    outliers = df[(df[col] < lower) | (df[col] > upper)]

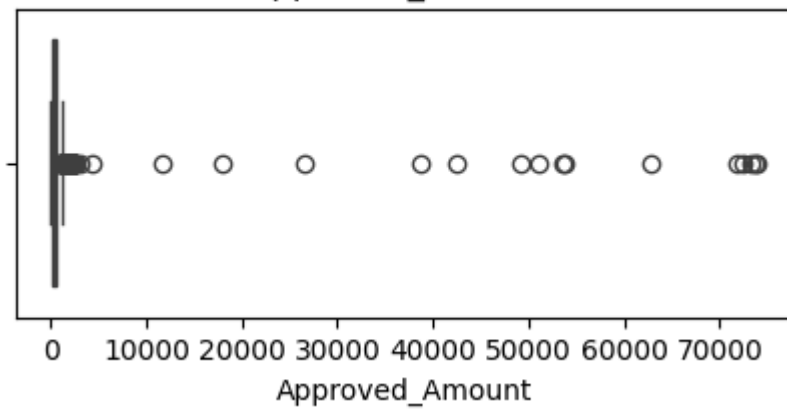
    print(f"{col}: {len(outliers)} outliers")
```


Patient_Age: 0 outliers
Claim_Amount: 439 outliers
Approved_Amount: 461 outliers
Prior_Visits_12m: 191 outliers
Length_of_Stay: 0 outliers
Days_Between_Service_and_Claim: 0 outliers
Number_of_Claims_Per_Provider_Monthly: 887 outliers

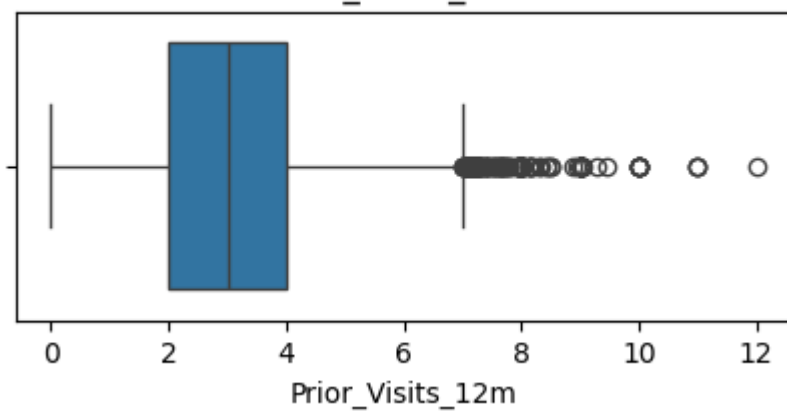
```
In [52]: for col in num_cols:
plt.figure(figsize=(5,2))
sns.boxplot(x=df[col])
plt.title(col)
plt.show()
```



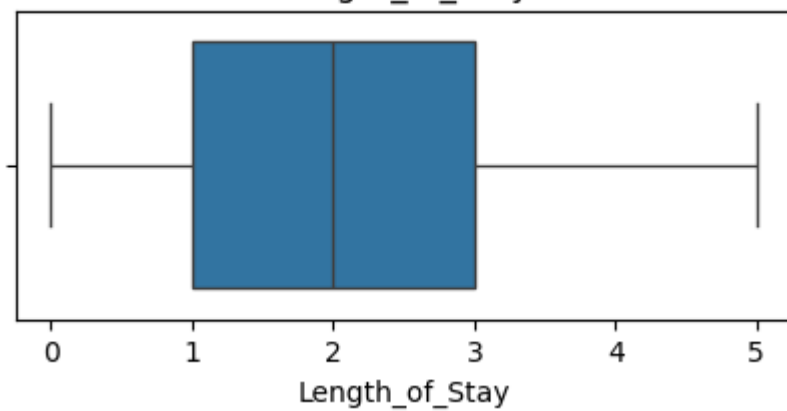
Approved_Amount

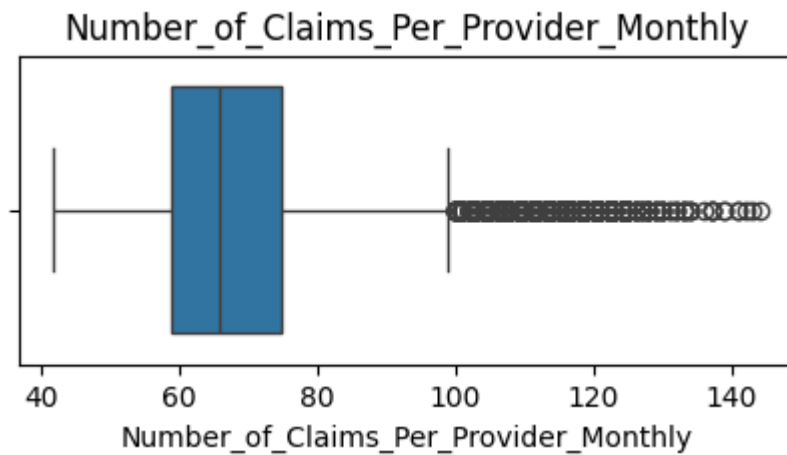
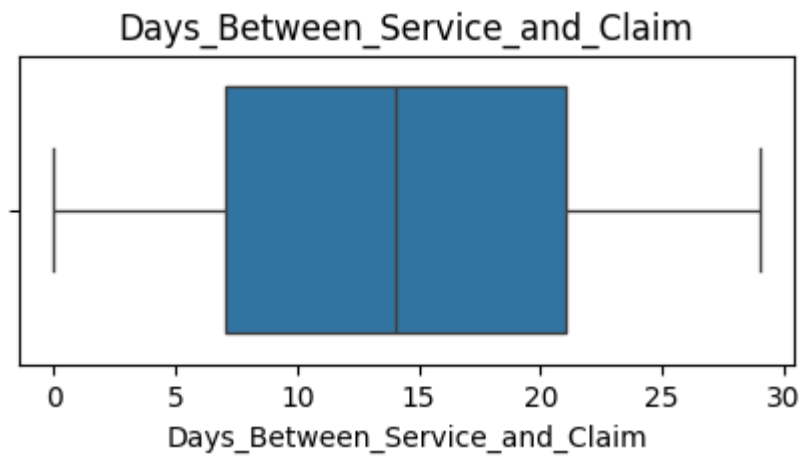


Prior_Visits_12m



Length_of_Stay





Outliers were identified using the IQR method. No outliers were removed because extreme values may represent genuine fraudulent activity and are therefore important for fraud detection analysis.

Fraud Drivers

Fraud Driver Analysis - Step 1

```
In [53]: df.groupby('Is_Fraud')['Claim_Amount'].agg([
    'count',
    'mean',
    'median',
```

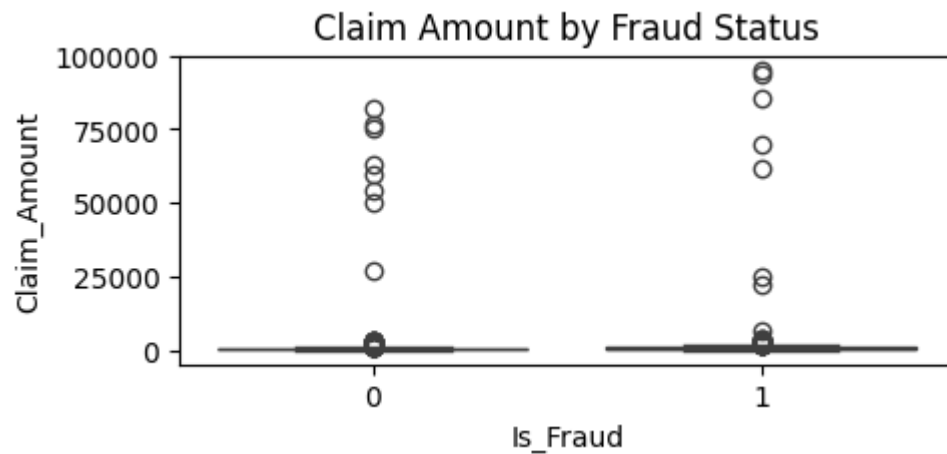
```
'min',
'max'
])
```

Out[53]:

	count	mean	median	min	max
Is_Fraud					

Is_Fraud	count	mean	median	min	max
0	17139	583.171846	484.59	60.0	81864.0
1	1625	1161.795058	788.31	60.0	95251.0

```
In [54]: plt.figure(figsize=(5,2))
sns.boxplot(x='Is_Fraud', y='Claim_Amount', data=df)
plt.title('Claim Amount by Fraud Status')
plt.show()
```



Fraudsters appear to target higher-value claims because larger claim amounts create greater financial gain while remaining hidden among thousands of legitimate healthcare transactions

Fraud Drivers Analysis - step 2

```
In [55]: df.groupby('Is_Fraud')['Approved_Amount'].agg([
'count',
'mean',
'median',
'min',
```

```
    'max'  
])
```

Out[55]:

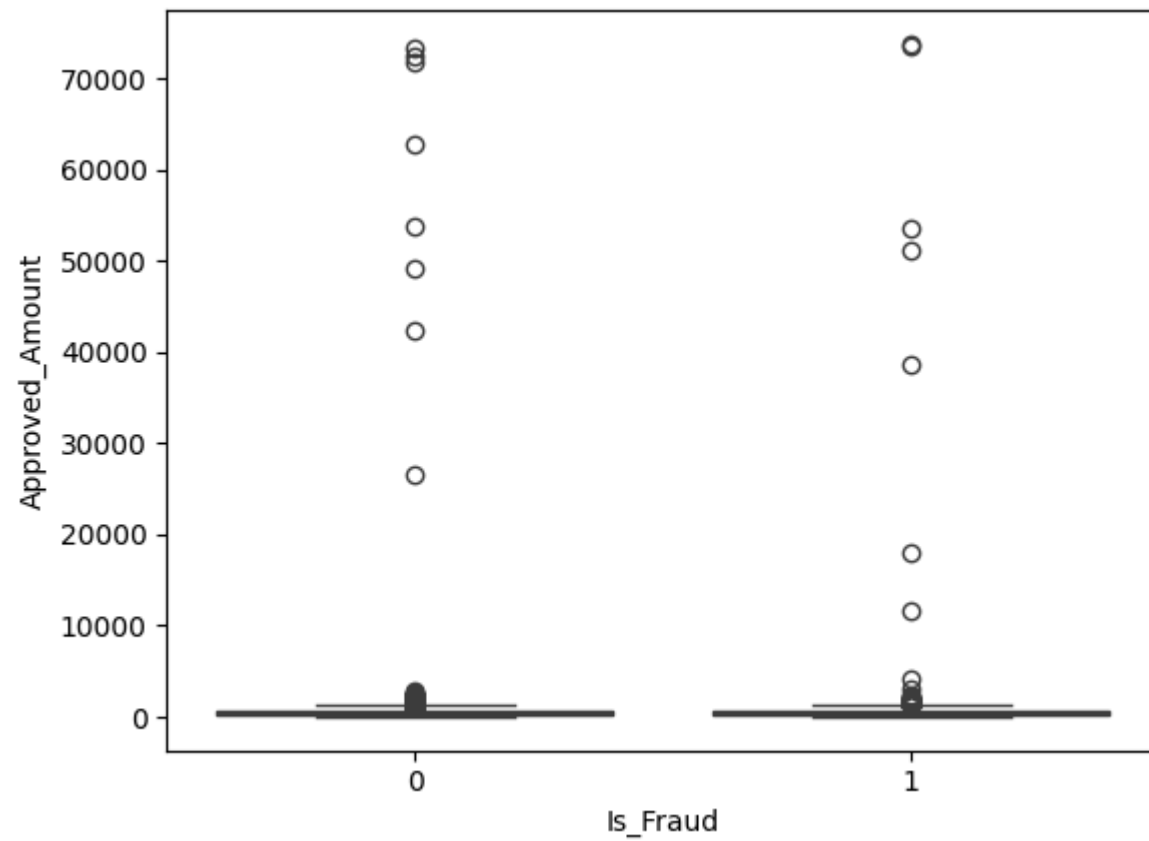
	count	mean	median	min	max
--	-------	------	--------	-----	-----

Is_Fraud

0	17139	511.642021	420.36	50.0	73384.60
---	-------	------------	--------	------	----------

1	1625	730.862868	475.13	50.0	73867.64
---	------	------------	--------	------	----------

```
In [56]: sns.boxplot(  
    x='Is_Fraud',  
    y='Approved_Amount',  
    data=df  
)  
plt.show()
```



The approval process appears to be reducing some of the financial impact of inflated claims.

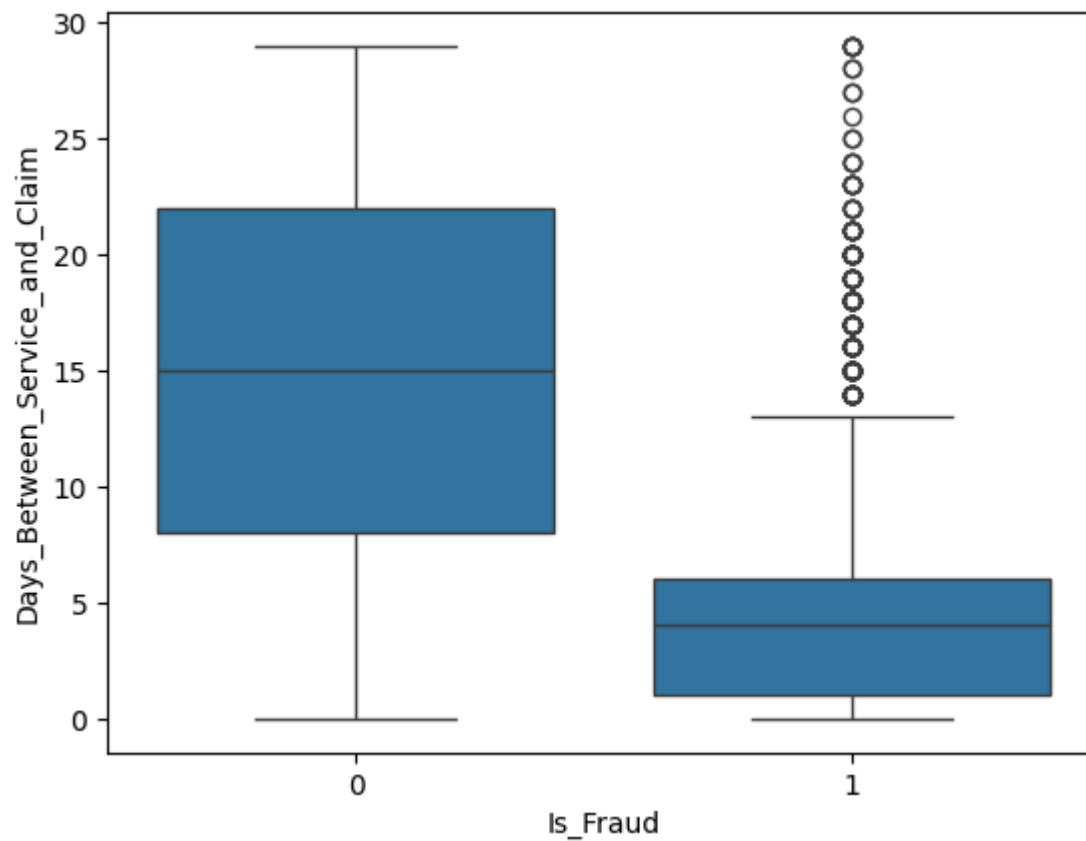
Fraud Driver Analysis - Step 3

```
In [59]: df.groupby('Is_Fraud')[  
         'Days_Between_Service_and_Claim'  
         ].describe()
```

```
Out[59]:
```

	count	mean	std	min	25%	50%	75%	max
Is_Fraud								
0	17139.0	15.077192	8.098367	0.0	8.0	15.0	22.0	29.0
1	1625.0	5.063385	5.399580	0.0	1.0	4.0	6.0	29.0

```
In [58]: sns.boxplot(  
         x='Is_Fraud',  
         y='Days_Between_Service_and_Claim',  
         data=df  
         )  
plt.show()
```



The typical fraud claim is submitted 11 days earlier than a legitimate claim.

Fraudsters want claims processed quickly.

Less time between service and submission means less opportunity for verification.

Automated or bulk fraudulent billing may be submitted immediately after service.

Fraud Driver Analysis - Step 4

```
In [60]: df.groupby('Is_Fraud')[  
          'Number_of_Claims_Per_Provider_Monthly'  
        ].describe()
```

Out[60]:

	count	mean	std	min	25%	50%	75%	max
Is_Fraud								
0	17139.0	67.993582	14.320872	42.0	59.0	66.0	74.0	144.0
1	1625.0	73.172923	18.385051	42.0	61.0	69.0	81.0	143.0

Unlike Submission Delay and Claim Amount, the separation here is modest.

This suggests:

High-volume providers are somewhat more likely to be linked to fraud.

However, provider volume alone is not a strong indicator.

Fraud appears to be driven more by claim behavior than by sheer provider activity.

Fraud Driver Analysis - Step 5

```
In [61]: df.groupby('Is_Fraud')[
          'Length_of_Stay'
        ].describe()
```

Out[61]:

	count	mean	std	min	25%	50%	75%	max
Is_Fraud								
0	17139.0	2.203629	1.703472	0.0	1.0	2.0	3.0	5.0
1	1625.0	2.154462	1.719962	0.0	1.0	2.0	3.0	5.0

Length of stay does not appear to be a meaningful fraud indicator in this dataset. Fraudulent and legitimate claims exhibit nearly identical hospitalization durations.

Fraud Driver Analysis - Step 6

```
In [62]: insurance_fraud = (
          df.groupby('Insurance_Type')['Is_Fraud']
```



```
.mean()  
.sort_values(ascending=False)*100  
)  
  
insurance_fraud
```

```
Out[62]: Insurance_Type  
Private      8.849162  
Medicare     8.827331  
Medicaid    8.616780  
Self-Pay     8.310932  
Name: Is_Fraud, dtype: float64
```

Fraud appears to be distributed relatively evenly across insurance programs. No single insurance type demonstrates a substantially elevated fraud rate.

This suggests:

Fraudsters are not targeting a specific insurance program.

Insurance type alone is unlikely to be a strong predictor of fraud.

Other factors such as claim amount, provider behavior, and submission timing are likely more important.

Fraud Driver Analysis - Step 7

```
In [63]: specialty_fraud = (  
    df.groupby('Provider_Specialty')['Is_Fraud']  
        .mean()  
        .sort_values(ascending=False)*100  
)  
  
specialty_fraud
```

```
Out[63]: Provider_Specialty  
General Practice    9.019948  
Orthopedics        8.937779  
Internal Medicine   8.709598  
Pulmonology         8.658009  
Cardiology          8.535133  
Neurology           7.891968  
Name: Is_Fraud, dtype: float64
```

Fraud is not concentrated within a single medical specialty. The relatively uniform fraud rates across specialties suggest that fraudulent behavior is more likely associated with claim characteristics and provider behavior than with the medical specialty itself.

Fraud Driver Analysis - Step 8

```
In [64]: visit_fraud = (  
    df.groupby('Visit_Type')['Is_Fraud']  
        .mean()  
        .sort_values(ascending=False)*100  
    )  
  
visit_fraud
```

```
Out[64]: Visit_Type  
Emergency      9.045710  
Inpatient       8.693509  
Outpatient      8.255742  
Name: Is_Fraud, dtype: float64
```

Emergency claims show a slightly higher fraud rate, possibly due to the urgency and complexity of emergency care. However, visit type alone does not appear to be a strong fraud predictor.

Hypothesis Test

H_0 (Null Hypothesis)

There is no significant difference in claim submission delay between fraudulent and non-fraudulent claims.

H_1 (Alternative Hypothesis)

There is a significant difference in claim submission delay between fraudulent and non-fraudulent claims.

```
In [65]: from scipy.stats import mannwhitneyu  
  
fraud = df[df['Is_Fraud']==1]['Days_Between_Service_and_Claim']  
non_fraud = df[df['Is_Fraud']==0]['Days_Between_Service_and_Claim']  
  
stat,p = mannwhitneyu(fraud, non_fraud)
```

```
print("Statistic:", stat)
print("P-value:", p)
```

Statistic: 4241369.5

P-value: 0.0

To validate whether claim submission behavior differs between fraudulent and legitimate claims, a Mann–Whitney U Test was performed on the **Days_Between_Service_and_Claim** variable. The null hypothesis stated that there is no significant difference in claim submission delays between the two groups, while the alternative hypothesis proposed that a significant difference exists. The test produced a statistic value of **4,241,369.5** and a **p-value of 0.0**, which is far below the significance threshold of 0.05. Therefore, the null hypothesis was rejected, confirming that claim submission delays differ significantly between fraudulent and legitimate claims. Further analysis revealed that legitimate claims have a median submission delay of **15 days**, whereas fraudulent claims have a median delay of only **4 days**. This finding suggests that fraudulent claims are typically submitted much more quickly after service, making rapid claim submission one of the strongest behavioral indicators of potential fraud in the dataset. From a business perspective, claims submitted shortly after service should be prioritized for fraud investigation, as they exhibit a significantly higher likelihood of fraudulent activity.

Risk Factor Discovery

Risk Factor 1: High Claim Amount

```
In [68]: df['Claim_Band'] = pd.qcut(
          df['Claim_Amount'],
          q=4,
          labels=['Low', 'Medium', 'High', 'Very High']
          )
          df['Claim_Band']
```

```

Out[68]: 0          Medium
         1          Medium
         2           High
         3           Low
         4          Medium
         ...
        18759         High
        18760         High
        18761      Very High
        18762      Very High
        18763      Very High
        Name: Claim_Band, Length: 18764, dtype: category
        Categories (4, object): ['Low' < 'Medium' < 'High' < 'Very High']

```

```

In [69]: df.groupby('Claim_Band')['Is_Fraud'].mean()*100

```

```

Out[69]: Claim_Band
Low          2.771264
Medium       4.945641
High         9.635472
Very High    17.288425
Name: Is_Fraud, dtype: float64

```

Claim amount is one of the strongest fraud risk factors in the dataset. As claim values increase, the likelihood of fraud rises substantially. Claims in the highest claim amount segment exhibit a fraud rate of 17.29%, compared to only 2.77% in the lowest segment. This suggests that high-value claims should receive enhanced scrutiny and may benefit from additional fraud screening controls.

Risk Factor 2: Submission Delay Band

```

In [70]: df['Delay_Band'] = pd.cut(
         df['Days_Between_Service_and_Claim'],
         bins=[-1,5,15,30],
         labels=['Fast','Medium','Slow']
         )

```

```

In [71]: df.groupby('Delay_Band')['Is_Fraud'].mean()*100

```

```
Out[71]: Delay_Band
Fast      30.411545
Medium     5.681133
Slow       1.311710
Name: Is_Fraud, dtype: float64
```

Claim submission speed is the most significant fraud risk factor identified in this analysis. Claims submitted within five days of service exhibit a fraud rate of 30.41%, compared to only 1.31% for claims submitted after fifteen days. This suggests that rapid claim submission is a strong behavioral indicator of potentially fraudulent activity and should be incorporated into fraud detection and claim review strategies.

Risk Factor 3: Provider Volume Band

```
In [72]: df['Provider_Volume_Band'] = pd.qcut(
        df['Number_of_Claims_Per_Provider_Monthly'],
        q=4,
        labels=['Low', 'Medium', 'High', 'Very High']
    )
```

```
In [73]: df.groupby('Provider_Volume_Band')['Is_Fraud'].mean()*100
```

```
Out[73]: Provider_Volume_Band
Low      7.100226
Medium   7.244163
High     7.668649
Very High 12.913386
Name: Is_Fraud, dtype: float64
```

Providers with very high monthly claim volumes demonstrate a fraud rate of 12.91%, nearly 1.8 times higher than low-volume providers. This suggests that unusually active providers may warrant additional review.

Provider Risk Segmentation

```
In [74]: ## Calculating Provider Fraud Rate
        provider_risk = (
            df.groupby('Provider_ID')
                .agg(
                    Total_Claims=('Is_Fraud', 'count'),
                    Fraud_Claims=('Is_Fraud', 'sum')
                )
        )
```

```
)  
  
provider_risk['Fraud_Rate'] = (  
    provider_risk['Fraud_Claims'] /  
    provider_risk['Total_Claims']  
) * 100  
  
provider_risk.sort_values(  
    'Fraud_Rate',  
    ascending=False  
) .head(20)
```

Out[74]:

	Total_Claims	Fraud_Claims	Fraud_Rate
Provider_ID			
P1026	1	1	100.0
P4567	2	2	100.0
P4945	1	1	100.0
P1032	1	1	100.0
P3173	1	1	100.0
P3855	1	1	100.0
P1305	1	1	100.0
P2514	1	1	100.0
P2337	1	1	100.0
P4825	1	1	100.0
P4284	1	1	100.0
P2726	1	1	100.0
P3013	1	1	100.0
P4961	1	1	100.0
P4891	1	1	100.0
P2523	1	1	100.0
P3705	1	1	100.0
P1338	1	1	100.0
P4591	1	1	100.0
P3393	1	1	100.0

```
In [76]: ### Creating a Risk Segment  
provider_risk['Risk_Segment'] = pd.cut(  
    provider_risk['Fraud_Rate'],
```

```
bins=[-1,5,10,20,100],
labels=[
    'Low Risk',
    'Medium Risk',
    'High Risk',
    'Critical Risk'
]
```

```
In [77]: ## Distribution of risk segment
provider_risk['Risk_Segment'].value_counts()
```

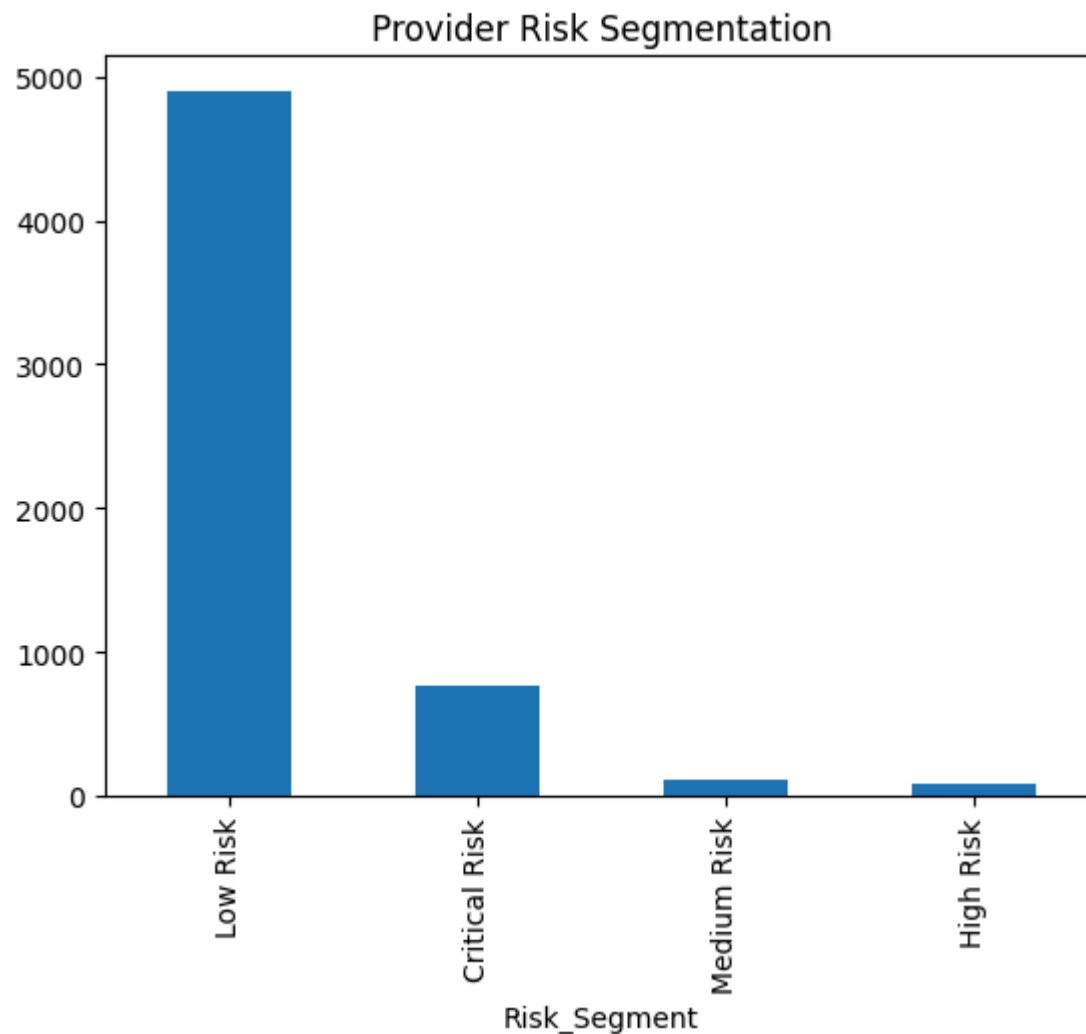
```
Out[77]: Risk_Segment
Low Risk      4901
Critical Risk   769
Medium Risk    115
High Risk       76
Name: count, dtype: int64
```

```
In [78]: ##Top 10 Riskiest Providers
provider_risk.sort_values(
    ['Fraud_Rate', 'Fraud_Claims'],
    ascending=False
).head(10)
```


Out[78]:

	Total_Claims	Fraud_Claims	Fraud_Rate	Risk_Segment
Provider_ID				
P2763	2	2	100.0	Critical Risk
P2782	2	2	100.0	Critical Risk
P4567	2	2	100.0	Critical Risk
P4655	2	2	100.0	Critical Risk
P5477	2	2	100.0	Critical Risk
P6575	2	2	100.0	Critical Risk
P6777	2	2	100.0	Critical Risk
P8669	2	2	100.0	Critical Risk
P9281	2	2	100.0	Critical Risk
P9427	2	2	100.0	Critical Risk

```
In [79]: provider_risk['Risk_Segment'].value_counts().plot(
          kind='bar'
        )
plt.title('Provider Risk Segmentation')
plt.show()
```



Provider risk segmentation revealed that while most providers demonstrate low levels of fraudulent activity, a substantial group of 769 providers falls into the Critical Risk category. These providers consistently exhibit elevated fraud rates and should be prioritized for enhanced monitoring, targeted audits, and detailed claim investigations. Concentrating investigative resources on this relatively small subset of providers can significantly improve fraud detection efficiency and reduce financial losses.

Patient Risk Segment

```
In [80]: ### Risk Factor 1 :: Prior Visit
```

```
df['Patient_Risk_Band'] = pd.qcut(
    df['Prior_Visits_12m'],
    q=4,
    labels=[
        'Low Utilization',
        'Medium Utilization',
        'High Utilization',
        'Very High Utilization'
    ]
)
```

```
In [81]: df.groupby('Patient_Risk_Band')['Is_Fraud'].mean()*100
```

```
Out[81]: Patient_Risk_Band
Low Utilization      8.842777
Medium Utilization   8.341980
High Utilization     8.359309
Very High Utilization 9.009877
Name: Is_Fraud, dtype: float64
```

```
In [83]: ##Risk Factor 2: Age Groups
df['Age_Group'] = pd.cut(
    df['Patient_Age'],
    bins=[0,18,35,50,65,100],
    labels=[
        '0-18',
        '19-35',
        '36-50',
        '51-65',
        '65+'
    ]
)
```

```
In [84]: df.groupby('Age_Group')['Is_Fraud'].mean()*100
```

```
Out[84]: Age_Group
0-18      7.455013
19-35     9.224760
36-50     9.051044
51-65     8.273580
65+       8.357021
Name: Is_Fraud, dtype: float64
```

```
In [85]: ## Risk Factor 3: Chronic Condition Status  
df.groupby('Chronic_Condition_Flag')['Is_Fraud'].mean()*100
```

```
Out[85]: Chronic_Condition_Flag  
0      8.646701  
1      8.693255  
Name: Is_Fraud, dtype: float64
```

Patient-level characteristics contribute very little to fraud risk in this dataset. Fraud appears to be driven primarily by claim behavior and provider-related factors rather than patient demographics or healthcare utilization patterns. This suggests that fraud detection efforts should focus on transactional indicators such as claim amount, submission timing, and provider activity rather than patient attributes.

Fraud Prediction Modeling

Build a machine learning model that can predict whether a healthcare claim is fraudulent or legitimate.

```
In [99]: ## Creating a Modeling Dataset  
model_df = df.copy()  
  
model_df = model_df.drop([  
    'Claim_ID',  
    'Provider_ID',  
    'Claim_Submission_Date'  
], axis=1)
```

```
In [100]: ## Separate Features and Target  
X = model_df.drop('Is_Fraud', axis=1)  
y = model_df['Is_Fraud']
```

```
In [101]: ##Checking Data Types  
X.dtypes.value_counts()
```

```
Out[101...  int64      6
float64     3
category     2
category     1
category     1
category     1
category     1
category     1
category     1
category     1
category     1
category     1
category     1
category     1
Name: count, dtype: int64
```

```
In [103...  ## Encoding Categorical
X = pd.get_dummies(
    X,
    drop_first=True
)
X.info()
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 18764 entries, 0 to 18763
```

```
Data columns (total 53 columns):
```

#	Column	Non-Null Count	Dtype
0	Patient_Age	18764 non-null	int64
1	Procedure_Code	18764 non-null	int64
2	Claim_Amount	18764 non-null	float64
3	Approved_Amount	18764 non-null	float64
4	Days_Between_Service_and_Claim	18764 non-null	int64
5	Number_of_Claims_Per_Provider_Monthly	18764 non-null	int64
6	Length_of_Stay	18764 non-null	int64
7	Chronic_Condition_Flag	18764 non-null	int64
8	Prior_Visits_12m	18764 non-null	float64
9	Patient_Gender_Male	18764 non-null	bool
10	Diagnosis_Code_E78.5	18764 non-null	bool
11	Diagnosis_Code_F41.9	18764 non-null	bool
12	Diagnosis_Code_I10	18764 non-null	bool
13	Diagnosis_Code_I25.10	18764 non-null	bool
14	Diagnosis_Code_J06.9	18764 non-null	bool
15	Diagnosis_Code_J18.9	18764 non-null	bool
16	Diagnosis_Code_K21.9	18764 non-null	bool
17	Diagnosis_Code_M54.5	18764 non-null	bool
18	Diagnosis_Code_N39.0	18764 non-null	bool
19	Insurance_Type_Medicare	18764 non-null	bool
20	Insurance_Type_Private	18764 non-null	bool
21	Insurance_Type_Self-Pay	18764 non-null	bool
22	Provider_Specialty_General Practice	18764 non-null	bool
23	Provider_Specialty_Internal Medicine	18764 non-null	bool
24	Provider_Specialty_Neurology	18764 non-null	bool
25	Provider_Specialty_Orthopedics	18764 non-null	bool
26	Provider_Specialty_Pulmonology	18764 non-null	bool
27	Patient_State_FL	18764 non-null	bool
28	Patient_State_GA	18764 non-null	bool
29	Patient_State_IL	18764 non-null	bool
30	Patient_State_NY	18764 non-null	bool
31	Patient_State_OH	18764 non-null	bool
32	Patient_State_PA	18764 non-null	bool
33	Patient_State_TX	18764 non-null	bool
34	Claim_Status_Pending	18764 non-null	bool
35	Claim_Status_Rejected	18764 non-null	bool
36	Visit_Type_Inpatient	18764 non-null	bool
37	Visit_Type_Outpatient	18764 non-null	bool
38	Claim_Band_Medium	18764 non-null	bool

39	Claim_Band_High	18764	non-null	bool
40	Claim_Band_Very High	18764	non-null	bool
41	Delay_Band_Medium	18764	non-null	bool
42	Delay_Band_Slow	18764	non-null	bool
43	Provider_Volume_Band_Medium	18764	non-null	bool
44	Provider_Volume_Band_High	18764	non-null	bool
45	Provider_Volume_Band_Very High	18764	non-null	bool
46	Patient_Risk_Band_Medium Utilization	18764	non-null	bool
47	Patient_Risk_Band_High Utilization	18764	non-null	bool
48	Patient_Risk_Band_Very High Utilization	18764	non-null	bool
49	Age_Group_19-35	18764	non-null	bool
50	Age_Group_36-50	18764	non-null	bool
51	Age_Group_51-65	18764	non-null	bool
52	Age_Group_65+	18764	non-null	bool

dtypes: bool(44), float64(3), int64(6)

memory usage: 2.1 MB

```
In [104... ## Train-Test Split
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)
```

```
In [105... ## Logistic Regression
from sklearn.linear_model import LogisticRegression

lr = LogisticRegression(
    class_weight='balanced',
    max_iter=1000,
    random_state=42
)

lr.fit(X_train, y_train)
```

```
Out[105... LogisticRegression ⓘ ?
  ▶ Parameters
```

```
In [107... ## Predictions
lr_pred = lr.predict(X_test)

lr_prob = lr.predict_proba(X_test)[: ,1]
```

```
In [108... ## Random Forest
from sklearn.ensemble import RandomForestClassifier

rf = RandomForestClassifier(
    n_estimators=200,
    class_weight='balanced',
    random_state=42
)

rf.fit(X_train, y_train)
```

```
Out[108... ▼ RandomForestClassifier ⓘ ?
► Parameters
```

```
In [109... rf_pred = rf.predict(X_test)

rf_prob = rf.predict_proba(X_test)[: ,1]
```

```
In [111... ## Evalutaion
from sklearn.metrics import (
    classification_report,
    confusion_matrix,
    roc_auc_score
)
```

```
In [112... ## Logistics Regression
print(classification_report(y_test, lr_pred))
print(confusion_matrix(y_test, lr_pred))
print("ROC-AUC:", roc_auc_score(y_test, lr_prob))
```


	precision	recall	f1-score	support
0	1.00	0.98	0.99	3428
1	0.86	0.98	0.92	325
accuracy			0.98	3753
macro avg	0.93	0.98	0.95	3753
weighted avg	0.99	0.98	0.98	3753

```
[[3375  53]
 [  6 319]]
ROC-AUC: 0.9987029889596984
```

In [113...

```
##Random Forest
print(classification_report(y_test, rf_pred))
print(confusion_matrix(y_test, rf_pred))
print("ROC-AUC:", roc_auc_score(y_test, rf_prob))
```

	precision	recall	f1-score	support
0	0.94	1.00	0.97	3428
1	0.98	0.31	0.47	325
accuracy			0.94	3753
macro avg	0.96	0.66	0.72	3753
weighted avg	0.94	0.94	0.93	3753

```
[[3426   2]
 [ 224 101]]
ROC-AUC: 0.9645664662059061
```

In [114...

```
## Feature Importance
importance = pd.DataFrame({
    'Feature': X_train.columns,
    'Importance': rf.feature_importances_
})

importance = importance.sort_values(
    'Importance',
    ascending=False
)

importance.head(15)
```

Out[114...

	Feature	Importance
4	Days_Between_Service_and_Claim	0.216022
3	Approved_Amount	0.124784
2	Claim_Amount	0.118653
42	Delay_Band_Slow	0.098634
5	Number_of_Claims_Per_Provider_Monthly	0.040600
35	Claim_Status_Rejected	0.035770
8	Prior_Visits_12m	0.031954
0	Patient_Age	0.031798
40	Claim_Band_Very_High	0.027814
41	Delay_Band_Medium	0.026613
1	Procedure_Code	0.021298
6	Length_of_Stay	0.016306
34	Claim_Status_Pending	0.014442
39	Claim_Band_High	0.011077
38	Claim_Band_Medium	0.008221

Two machine learning models were evaluated for healthcare fraud detection: Logistic Regression and Random Forest. Logistic Regression achieved an accuracy of 98%, a fraud recall of 98%, an F1-score of 92%, and an ROC-AUC score of 0.9987. In comparison, Random Forest achieved an accuracy of 94% and a higher fraud precision of 98%, but its fraud recall dropped significantly to 31%, resulting in an F1-score of only 47%. While Random Forest generated fewer false positives, it failed to identify 224 fraudulent claims, making it unsuitable for a fraud detection environment where missed fraud cases can lead to substantial financial losses. Logistic Regression successfully detected 319 out of 325 fraudulent claims while maintaining strong overall performance. Based on these results, Logistic Regression was selected as the final model due to its superior ability to identify fraudulent claims, excellent recall, and near-perfect ROC-AUC performance.